Hochschule Hamm-Lippstadt

Systems Engineering & Prototyping

# Autonomous 2WD Robot Vehicle

A Hybrid Line-Following and Obstacle-Avoiding System

*Final Milestone — Project Submission*

**Group:** D3

**Team:** Md Afif Hasan
Rei Halilaj
Ambrose
Jubayer Ahmed

**Supervisor:** Faezeh Pasandideh

**Date:** July 2026

Repository: github.com/HASAN-AFIF720/Prototyping-D3

# Abstract

This report documents the design, modelling, and implementation of an autonomous, untethered two-wheel-drive (2WD) robot vehicle developed as a Systems Engineering prototyping project. The vehicle combines two behaviours on a single Arduino UNO: infrared (IR) line following for navigation and ultrasonic obstacle avoidance for safety. A strict priority hierarchy governs the control loop — obstacle avoidance always pre-empts line tracking. The work is grounded in a formal SysML model (requirement, block, use case, sequence, activity, and package diagrams) that traces every hardware block back to a defined requirement. The prototype was assembled on a custom 3D-modelled chassis and validated through incremental bench testing. This document presents the full engineering process from requirements to a tested prototype, satisfying both project milestones: lane following (Milestone 1) and obstacle avoidance (Milestone 2).

# Contents

# 1 Introduction

## 1.1 Motivation

Autonomous driving is a key enabler for the future of mobility, relying on the coordinated operation of perception, decision-making, and actuation. Autonomous routing, obstacle detection, and parking are representative capabilities of such systems. This project miniaturises that challenge into a tabletop robot: a vehicle that must perceive a track and its surroundings, decide on a safe action, and actuate its motors accordingly — all without human intervention or a physical tether.

## 1.2 Objectives and Scope

The system was required to autonomously drive along a defined track using line detection while detecting and avoiding obstacles in its path. The scope of the final milestone comprises:

- A complete SysML systems-engineering model.
- A physically assembled, untethered prototype.
- Embedded firmware implementing the priority-based control logic.
- Incremental validation of both behaviours on hardware.

## 1.3 Milestones

The project was structured around two deliverable milestones:

- **Milestone 1 — Lane Following:** the vehicle follows the track via IR line detection.
- **Milestone 2 — Obstacle Avoidance:** ultrasonic obstacle avoidance layered on the confirmed line-following base.

# 2 Requirements Engineering

The analysis began by capturing system requirements and mapping each to the hardware block that satisfies it. Figure 1 shows the SysML requirement diagram: a top-level autonomous-operation requirement decomposes into obstacle avoidance, line tracking, untethered power, and a nested safety threshold.

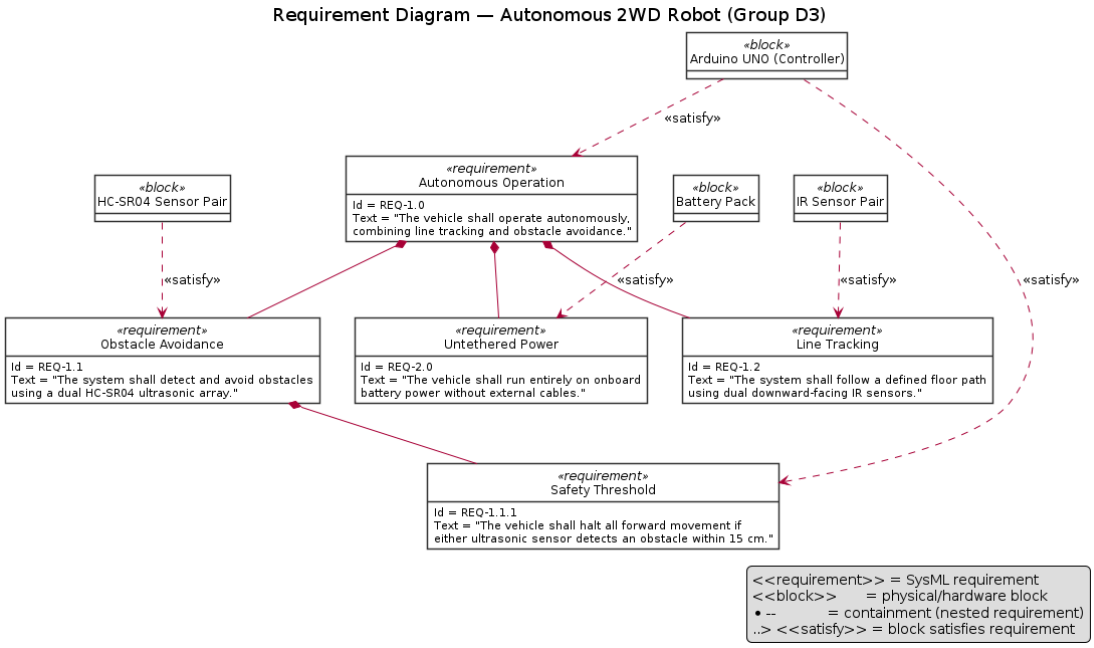


**Figure 1:** SysML Requirement Diagram with block `satisfy` relationships.

**Table 1:** Requirements traceability.

| ID | Requirement | Statement | Satisfied by |
|---|---|---|---|
| REQ-1.0 | Autonomous Operation | Operate autonomously, combining line tracking and obstacle avoidance. | Arduino UNO |
| REQ-1.1 | Obstacle Avoidance | Detect and avoid obstacles using a dual HC-SR04 ultrasonic array. | HC-SR04 pair |
| REQ-1.1.1 | Safety Threshold | Halt all forward movement if either sensor detects an obstacle within 15 cm. | Arduino + HC-SR04 |
| REQ-1.2 | Line Tracking | Follow a defined floor path using dual downward-facing IR sensors. | IR sensor pair |
| REQ-2.0 | Untethered Power | Run entirely on onboard battery power without external cables. | Battery pack |

# 3 System Architecture

## 3.1 Hardware Composition

The vehicle is composed of a controller, a motor driver, two actuators, two sensor pairs, a battery, and a custom chassis. The Block Definition Diagram (Figure 2) captures this composition and its multiplicities.

**Table 2:** Bill of materials.

| Component | Part | Role |
|---|---|---|
| Microcontroller | 1× Arduino UNO (5V logic) | Central decision maker |
| Motor Driver | 1× L298N dual H-bridge | PWM motor control (ENA/ENB jumpers removed) |
| Actuators | 2× DC gear motors | Left/right differential drive |
| Obstacle Sensors | 2× HC-SR04 | Front-left / front-right distance |
| Tracking Sensors | 2× IR line sensors | Downward-facing line detection |
| Power | 12V / AA battery pack | Motor power isolated from logic |
| Chassis | Custom 3D-modelled parts | Mounts and structure |

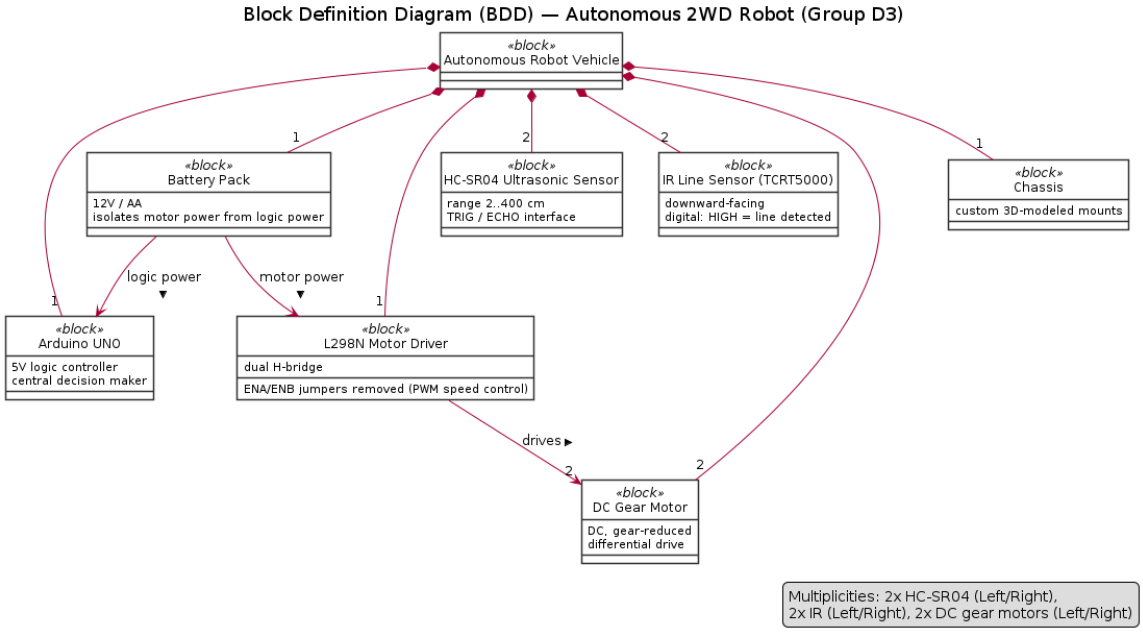


**Figure 2:** Block Definition Diagram (BDD) — system composition and multiplicities.

## 3.2 Internal Structure and Pin Allocation

The Internal Block Diagram (Figure 3) details the power and signal connectors between blocks, annotated with the Arduino pin allocation summarised in Table 3.

**Table 3:** Pin map (as flashed).

| Subsystem | Signal | Arduino Pin |
|---|---|---|
| IR line sensor — Left | digital | D12 |
| IR line sensor — Right | digital | D13 |
| Right motor | ENA (PWM) / IN1 / IN2 | D10 / D5 / D4 |
| Left motor | ENB (PWM) / IN3 / IN4 | D11 / D7 / D6 |
| Ultrasonic — Left | TRIG / ECHO | A2 / A3 |
| Ultrasonic — Right | TRIG / ECHO | D2 / D3 |

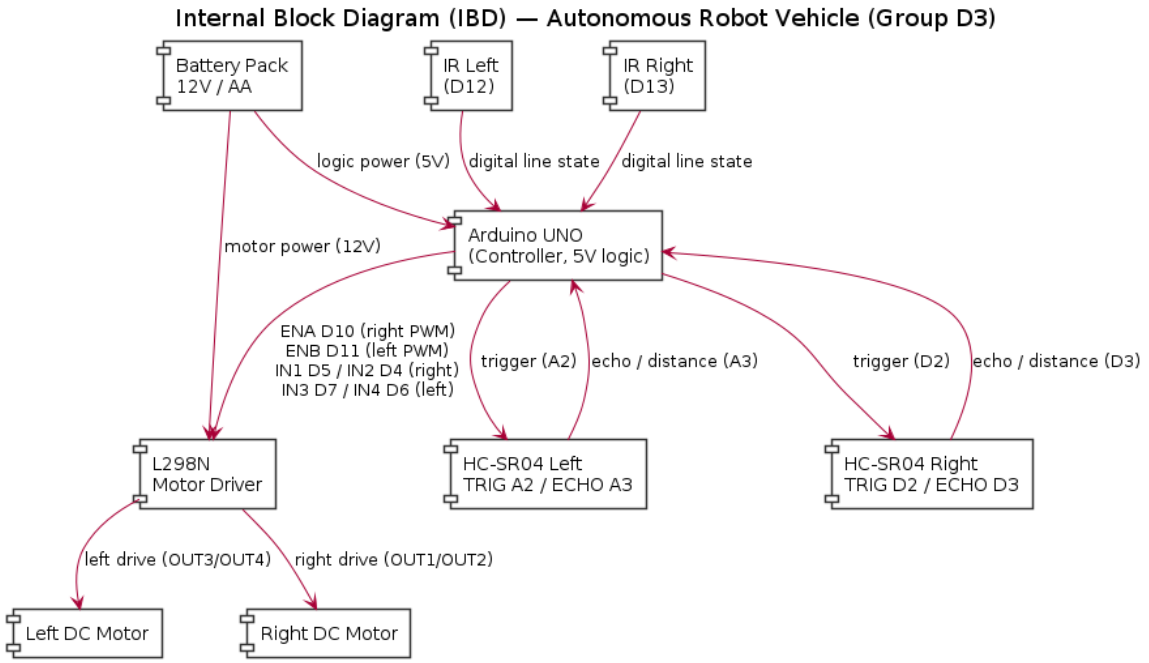


**Figure 3:** Internal Block Diagram (IBD) — pin-level connectors.

## 3.3 Circuit / Wiring

Figure 4 shows the full wiring schematic. Power flows from the battery to the L298N (12V) and, via the Arduino's regulated 5V rail, to the sensors; a common ground ties the driver, controller, and sensors together.

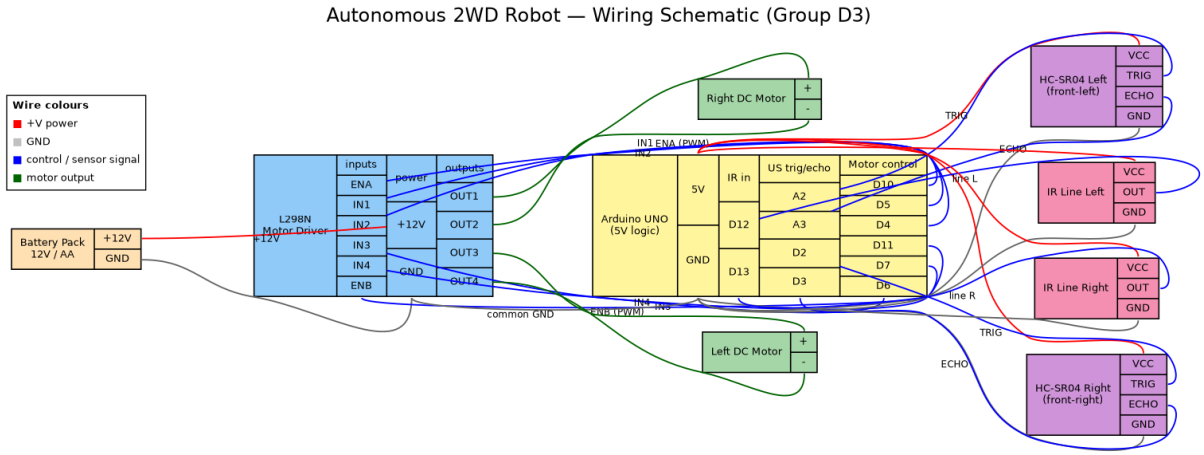


**Figure 4:** Circuit wiring schematic (colour-coded: power, ground, signal, motor output).

# 4 Behavioural Modelling

## 4.1 Use Cases

The operator interacts with the system through three use cases — power-on, start navigation, and emergency stop. Navigation `includes` both line tracking and obstacle scanning, which in turn `extend` into steering corrections and evasive turns (Figure 5).

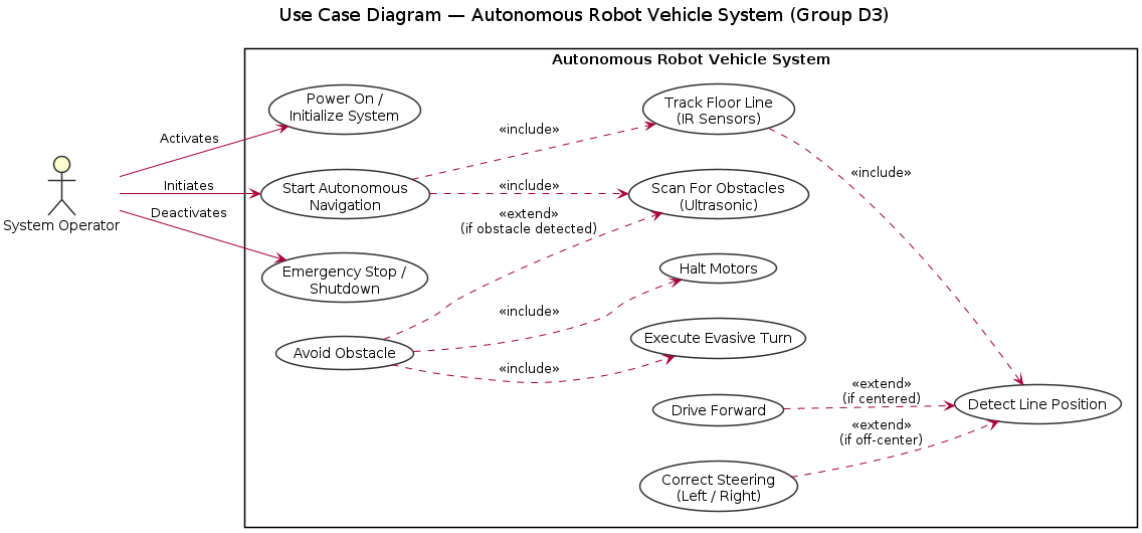


**Figure 5:** Use Case Diagram.

## 4.2 Runtime Sequence

The Sequence Diagram (Figure 6) traces one navigation loop: the controller triggers both ultrasonic sensors, and either enters the safety branch (halt, compare, evasive turn with a motor-power request) or the navigation branch (read IR, drive/correct).

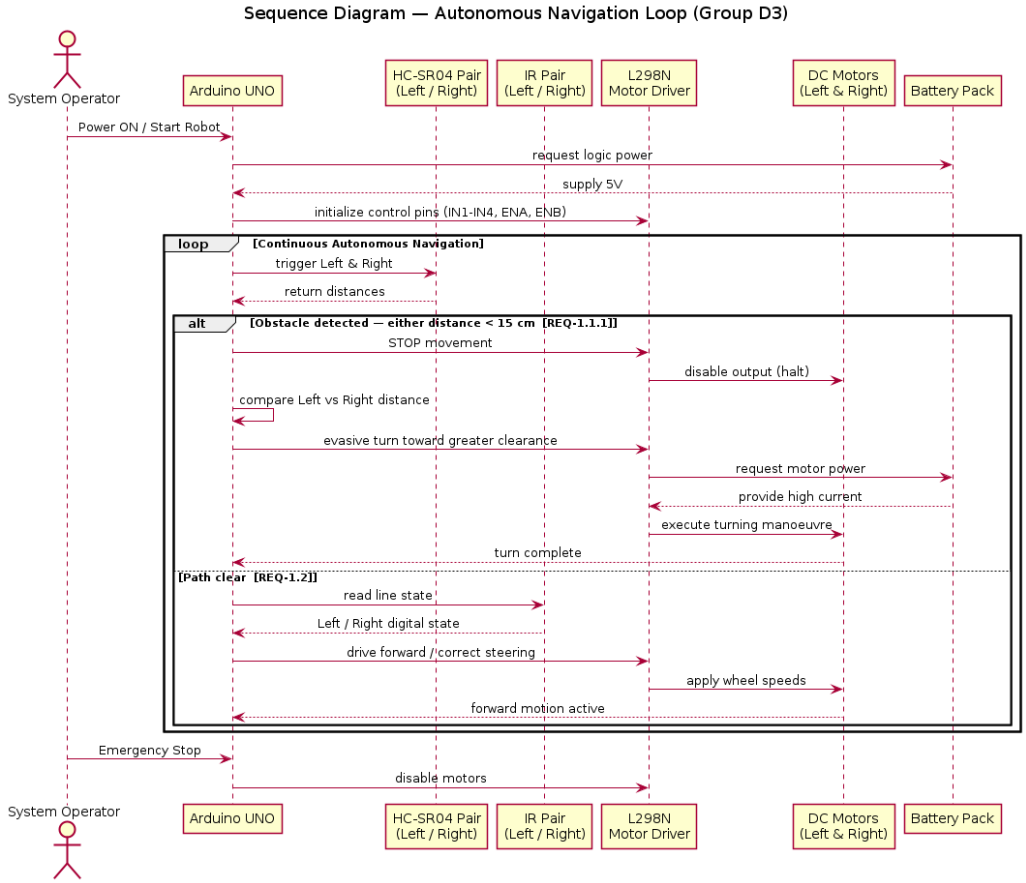


**Figure 6:** Sequence Diagram — autonomous navigation loop.

## 4.3 Control Activity

The Activity Diagram (Figure 7) expresses the `loop()` as an if/else flowchart with the safety-first gate.

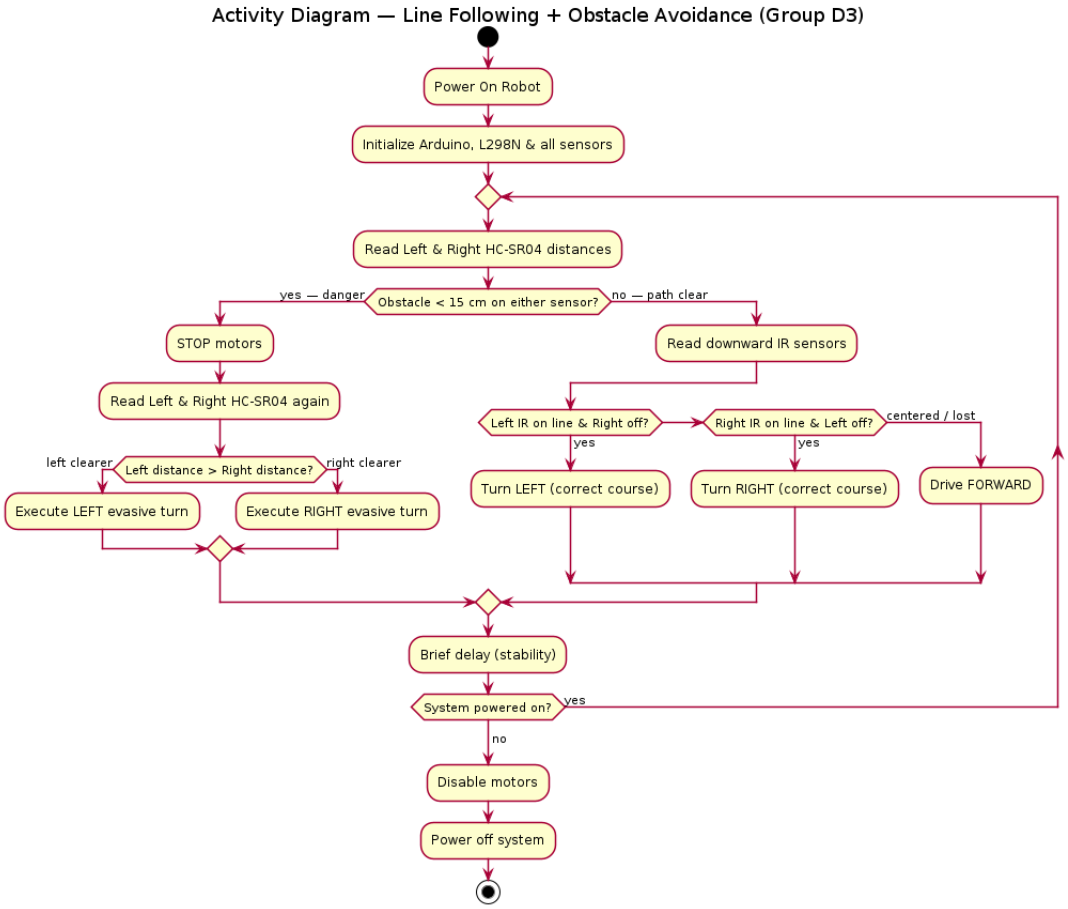


**Figure 7:** Activity Diagram — line following and obstacle avoidance.

## 4.4 Software Architecture

The Package Diagram (Figure 8) separates the Arduino core library from the project's configuration, drivers, control logic, and main packages.

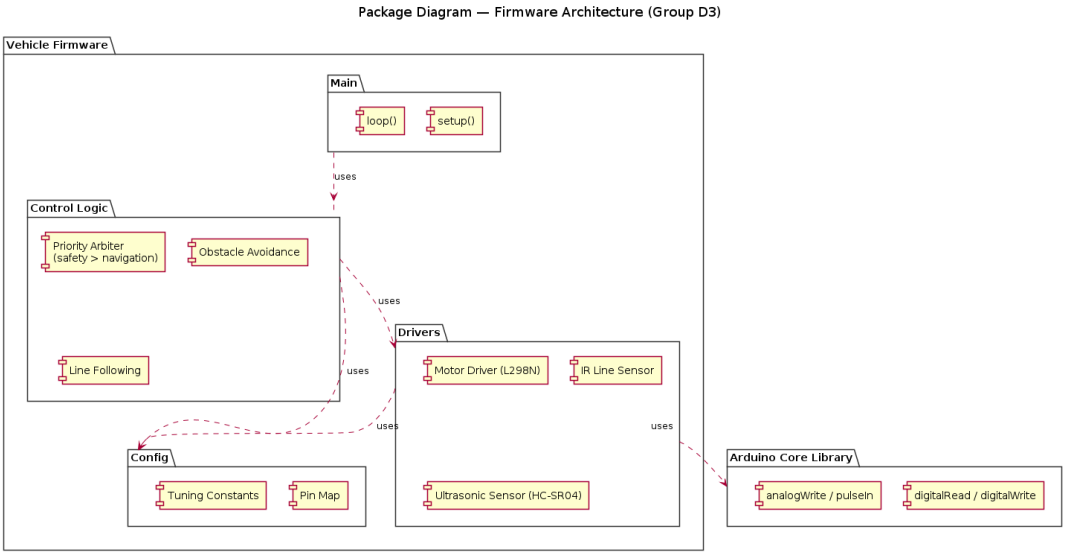


**Figure 8:** Package Diagram — firmware architecture.

## 5 Control Algorithm

The firmware realises a *Sense* → *Think* → *Act* pipeline with a two-tier priority hierarchy. Safety (obstacle avoidance) is evaluated first every cycle; navigation (line tracking) runs only when the path ahead is clear.

1. **Priority 1 — Obstacle Avoidance.** Trigger both HC-SR04 sensors. If *either* distance is below the 15 cm threshold (REQ-1.1.1), halt the motors, compare the left and right distances, and execute an evasive turn toward the side with the greater open distance.

2. **Priority 2 — Line Tracking.** If the path is clear, read the IR sensors and correct course: turn toward the sensor that detects the line, or drive straight when centred.

## 6 Firmware Implementation

The firmware is written in Arduino C++ and organised into a low-level motor driver, sensor read routines, and the priority-gated main loop. Listing 1 shows the core control loop with SysML traceability comments; Listing 2 shows the signed-speed motor driver.

```
// Sense -> Think -> Act, safety before navigation
void loop() {
  long dL = readUltrasonic(US_LEFT_TRIG,  US_LEFT_ECHO);
  long dR = readUltrasonic(US_RIGHT_TRIG, US_RIGHT_ECHO);

  // ---- Priority 1: Obstacle Avoidance (REQ-1.1, REQ-1.1.1) ----
  if (dL < OBSTACLE_CM || dR < OBSTACLE_CM) {
    stopMotors();                  // halt on detection
    if (dL >= dR) pivotLeft();     // turn toward greater clearance
    else          pivotRight();
    return;                        // skip navigation this cycle
  }

  // ---- Priority 2: Line Tracking (REQ-1.2) ----
  int irLeft  = digitalRead(IR_LEFT);
  int irRight = digitalRead(IR_RIGHT);
  if      (irLeft == LINE && irRight != LINE)  turnLeft();
  else if (irRight == LINE && irLeft != LINE)  turnRight();
```

```
  else                                              driveForward();
}
```

**Listing 1:** Priority-gated main loop (representative).

```
// positive = forward, negative = reverse, 0 = brake
void rotateMotor(int rightSpeed, int leftSpeed) {
  digitalWrite(rightIn1, rightSpeed > 0);
  digitalWrite(rightIn2, rightSpeed < 0);
  digitalWrite(leftIn1,  leftSpeed  > 0);
  digitalWrite(leftIn2,  leftSpeed  < 0);
  analogWrite(enableRight, abs(rightSpeed));
  analogWrite(enableLeft,  abs(leftSpeed));
}
```

**Listing 2:** Signed-speed motor driver over the L298N.

A key implementation lesson concerned timer configuration: the PWM enable pins are governed by Timer1 and Timer2, so the Arduino default Timer0 must be left untouched — modifying it silently distorts `delay()` and `pulseIn()`, which the HC-SR04 timing depends on.

## 7 Testing and Results

Each behaviour was validated in isolation before integration, following the principle that a confirmed-working base is extended rather than rewritten. Table 4 summarises the outcomes.

**Table 4:** Validation summary.

| Test | Criterion | Result |
|---|---|---|
| IR polarity check | Correct line vs. surface reading | Pass |
| Straight-line tracking | Follows track without drift | Pass |
| Curve handling | Negotiates corners without departure | Pass |
| Obstacle halt | Stops within 15 cm threshold | Pass |
| Evasive turn | Turns toward clearer side | Pass |
| Return to line | Resumes tracking after avoidance | Pass |

Tuning focused on motor speeds: full speed on straights with automatic speed reduction on curves, and a hard-reverse inner wheel (at roughly 80% of the curve speed) as the middle ground between soft turns (which caused track departure) and full-reverse turns (which caused skid).

# 8 Challenges and Solutions

**Table 5:** Key challenges and how they were addressed.

| Challenge | Solution |
|---|---|
| Choosing an effective chassis design | Evaluated multiple CAD designs before building. |
| Tangled / loose wiring during testing | Re-checked, labelled, and secured connections. |
| Faulty Arduino and ultrasonic modules | Replaced the affected controller and sensors. |
| Inverted IR line logic (recurring bug) | Bench-verified sensor polarity before building on new code. |
| Ultrasonic timing corruption | Left Timer0 at default; used Timer1/Timer2 for PWM. |
| Inconsistent multi-sensor behaviour | Debugged code in small modules before integration. |

# 9 Project Management

The team used a shared Git repository with continuous uploads of results and responsibilities. The overall task was divided per member by strengths across mechanical design, sensor logic, algorithm, and integration. Version numbers tracked firmware states so that a confirmed-working baseline was always preserved before new features were layered on top.

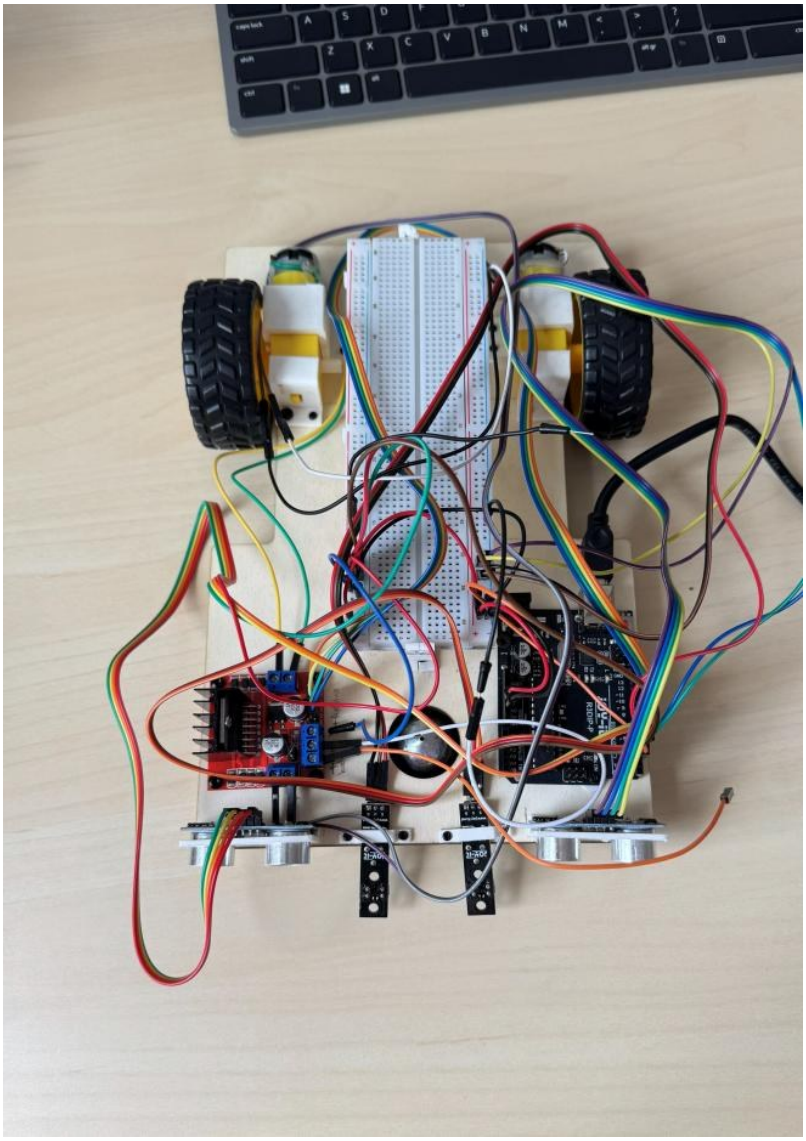

**Figure 9:** Top-down view of the assembled prototype showing sensor and module placement.

## 10 Conclusion and Future Work

The project delivered a fully modelled and physically realised autonomous 2WD vehicle that satisfies both milestones: reliable lane following and safety-first obstacle avoidance. The SysML model provided end-to-end traceability from requirements to firmware, and incremental bench testing confirmed correct behaviour. Future work could include a PID-based line controller for smoother tracking, a third forward sensor to reduce the blind spot between the two ultrasonic cones, and closed-loop speed control using wheel encoders.

## Team Contributions

| Member | Primary Contribution |
|---|---|
| Md Afif Hasan | Scrum backlog, Hardware architecture, integration |
| Rei Halilaj | Sensor logic (IR and ultrasonic), Control algorithm and firmware |
| Jubayer Ahmed | Mechanical design, firmware, GitHub and documentation |
| Ambrose | Testing, challenge analysis |

## References

[1] Object Management Group, *OMG Systems Modeling Language (SysML) Specification*.

[2] Arduino, *Arduino UNO Rev3 Documentation*, arduino.cc.

[3] *HC-SR04 Ultrasonic Ranging Module Datasheet*.

[4] STMicroelectronics, *L298 Dual Full-Bridge Driver Datasheet*.